



Testing Agents and their Tools:

Offline Evaluation, Synthetic Tasks and A/B experiments



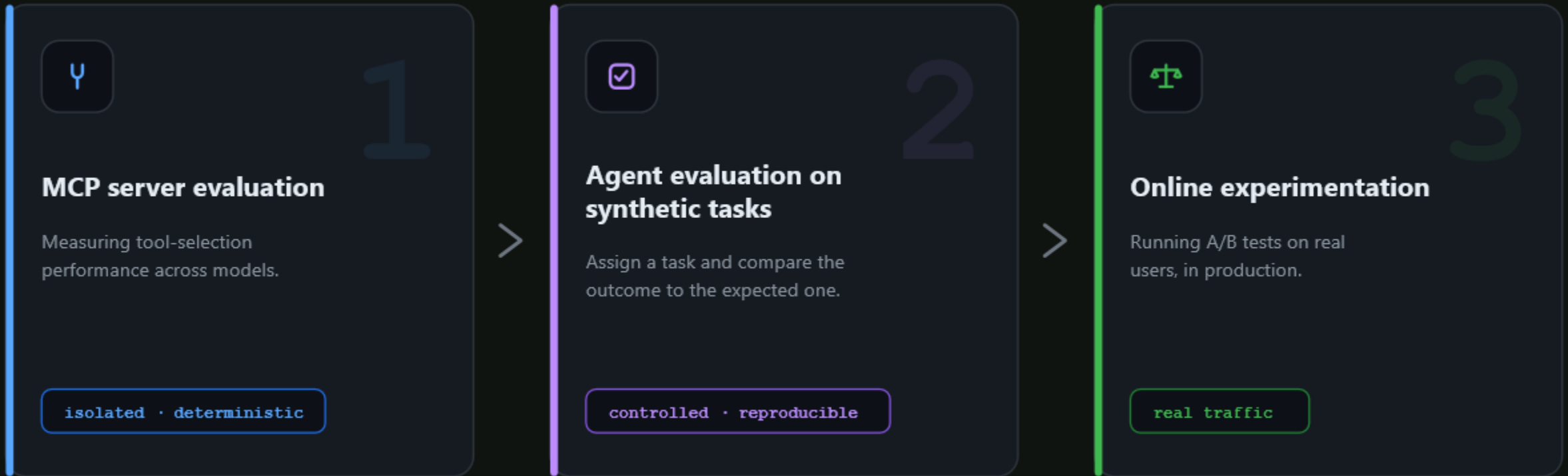
KSENIA BOBROVA
Senior Software Engineer
GitHub

Same task – different approaches

Which one is “correct”?



Three layers of evaluation



Curated dataset for tool selection

EACH BENCHMARK ITEM CONTAINS

- 1 Input**
a request in natural language
- 2 Expected tools**
tools we expect to be called
- 3 Expected arguments**
arguments passed to each tool

Counting issues in a time period

INPUT

How many issues were created in
github/github-mcp-server during April 2025?

EXPECTED TOOL

`list_issues`

EXPECTED ARGUMENTS

```
owner: github
repo: github-mcp-server
since: 2025-04-01T00:00:00Z
```

Merging a pull request

INPUT

Merge PR 123 in github/docs using squash
merge with title "Update installation guide"

EXPECTED TOOL

`merge_pull_request`

EXPECTED ARGUMENTS

```
owner: github
repo: docs
pullNumber: 123
merge_method: squash
commit_title: Update installation guide
```



Pipeline stages

1 Fulfillment

Run each benchmark across multiple models, recording which tools were called.

RECORDS

Invoked tools + the arguments supplied on every run.

2 Evaluation

Process the raw outputs to compute metrics and scores.

METRICS · PER TOOL

Argument-match scores that grade tool-call correctness.

3 Summarization

Aggregate dataset-level stats into the final evaluation report.

METRICS · WHOLE DATASET

Precision · Recall
Accuracy · F1-score



Tool selection as... classification?

Reading the `list_issues` vs `search_issues` confusion

Benchmark set

20 single-tool benchmarks

● 10 expect `search_issues`

● 10 expect `list_issues`

The model mislabels

3 `search` → `list`

Confusion matrix

Rows = expected tool · Columns = tool the model actually called

		Called by model →	
		<code>search_issues</code>	<code>list_issues</code>
<code>search_issues</code>		7	3
<code>list_issues</code>		0	10

3

These 3 are the whole story:
they lower `search_issues` recall and `list_issues` precision.

Precision (`list_issues`)

Of all `list_issues` calls, how many were correct?

$$10 / (10 + 3) =$$

0.77

Recall (`search_issues`)

Of all expected `search_issues`, how many were found?

$$7 / 10 =$$

0.70

Accuracy (whole dataset)

Share of inputs that produced the expected tool call.

$$(7 + 10) / 20 =$$

0.85



Learn more about GitHub MCP evals



<https://gh.io/AA13jm01>

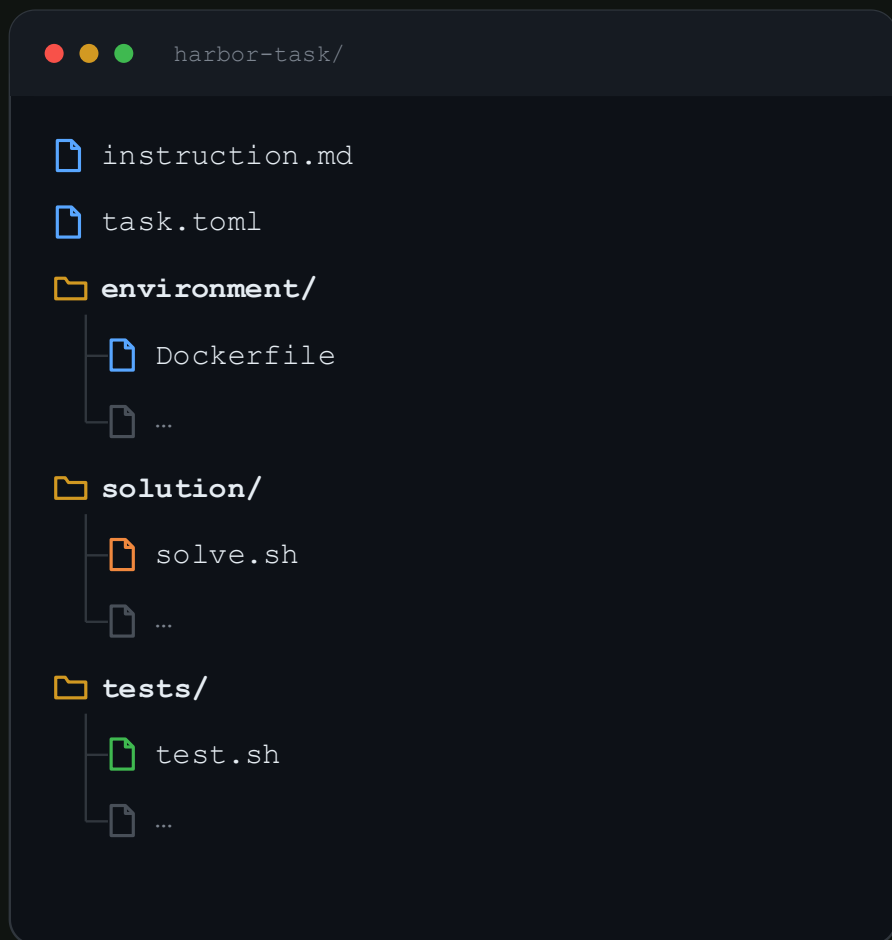
Run agent on synthetic tasks

Controlled, repeatable challenges we build to measure the agent.



Oops... ❌

Tasks in harbor format



■ **instruction.md**

The task, written in natural language, that the agent must complete.

■ **task.toml**

Task config & metadata — setup, timeouts, and scoring weights.

■ **environment/**

The `Dockerfile` and files that build the sandbox the task runs in.

■ **solution/**

Reference "oracle" solution.

`solve.sh` produces the correct result.

■ **tests/**

`test.sh` verifies the agent's work and decides pass / fail.



Run, capture, evaluate



Metrics



Correctness

Did the agent actually solve the task — judged by checks and LLM graders.



Tool use

Which tools it chose and how often — and how often those calls succeeded.



Cost & tokens

How much it consumed — tokens and spend, including savings from caching.



Speed

How quickly it responded — total time, time to first token, per-step latency.



Code changes

Size and shape of the edits — lines and files touched, plus over-deletion flags.

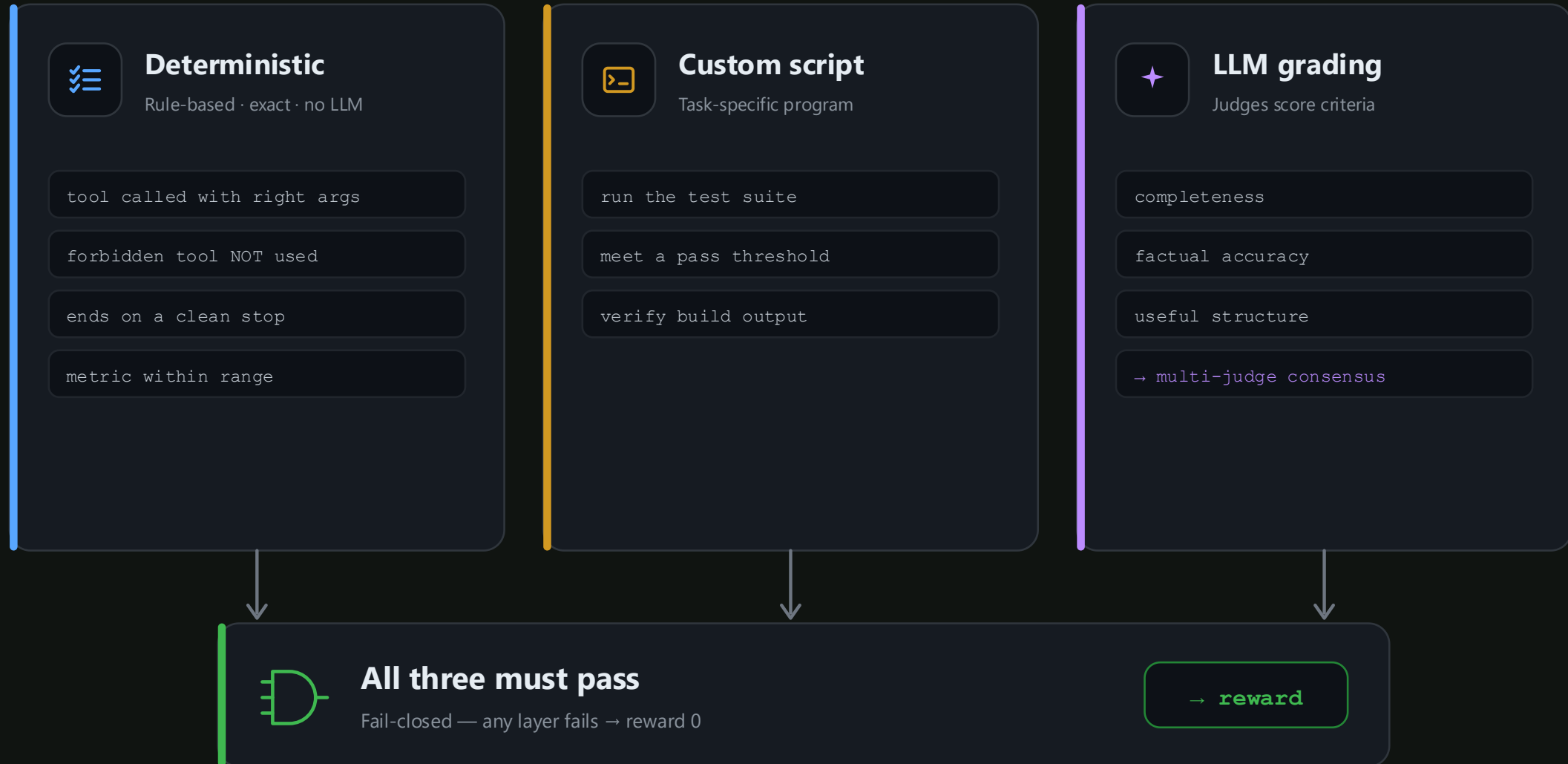


Reliability

Signs of trouble — loops, empty results, errors, and wasted effort.



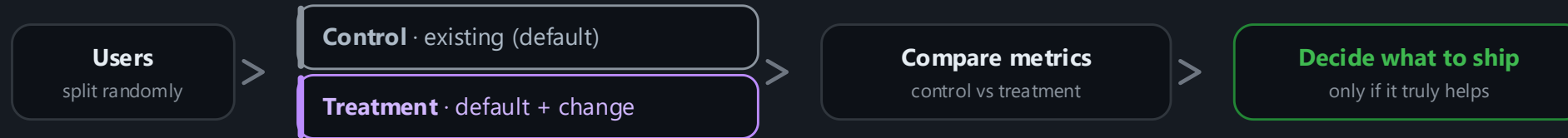
Defining a successful run



Agent online evaluation

A/B test — a controlled experiment

The scientific way to prove **causality**: the change in metrics is caused by the treatment, not by chance.



🎯 **What we expect:** a positive effect — or simply no negative effect (a guardrail), depending on the experiment's design.

? The two hypotheses

$$\text{Treatment} - \text{Control} = \Delta$$

e.g. 12.4% - 11.9% = +0.5%

Alternative hypothesis

the change caused the difference

Null hypothesis

the difference is just random chance



Microsoft Experimentation Platform

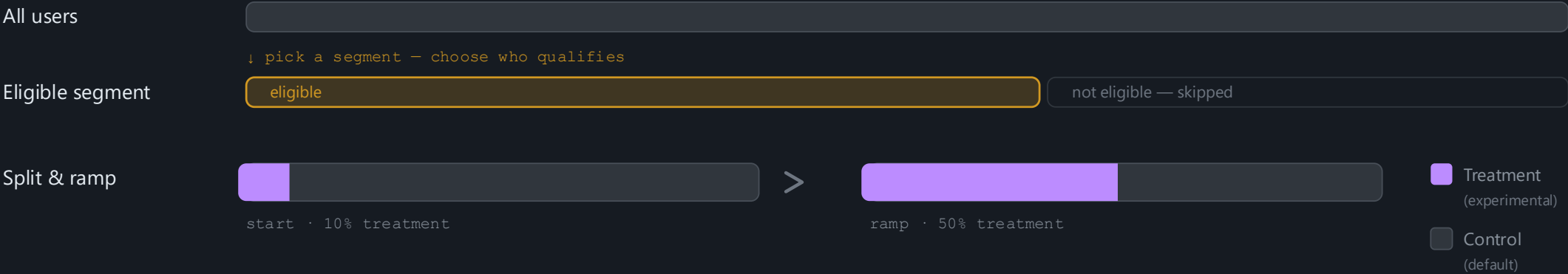


<https://gh.io/AA13hgua>



Experiment rollout

HOW THE ROLLOUT WORKS



HOW IT LOOKS IN CODE

FEATURE FLAG	VALUE	CODE PATH	ASSIGNMENT
F featureEnabled	True	if (featureEnabled == true) { runWithFeature() }	Treatment
	False	if (featureEnabled == false) { runDefault() }	Control



Computing online metrics

- 1

Raw signal data
one row per event
- 2

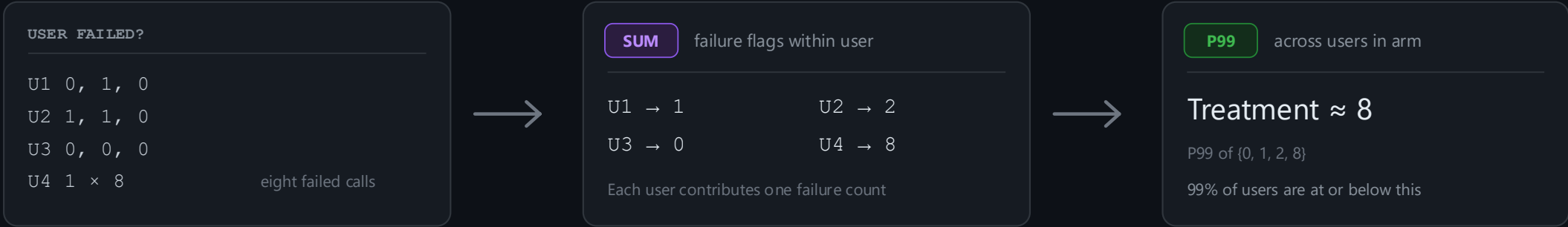
Aggregation per unit
one row per user
- 3

Statistical aggregation
one result per arm

METRIC 1 Average tool-definition token count per user



METRIC 2 99th percentile of tool-call failures per user



P-values

p

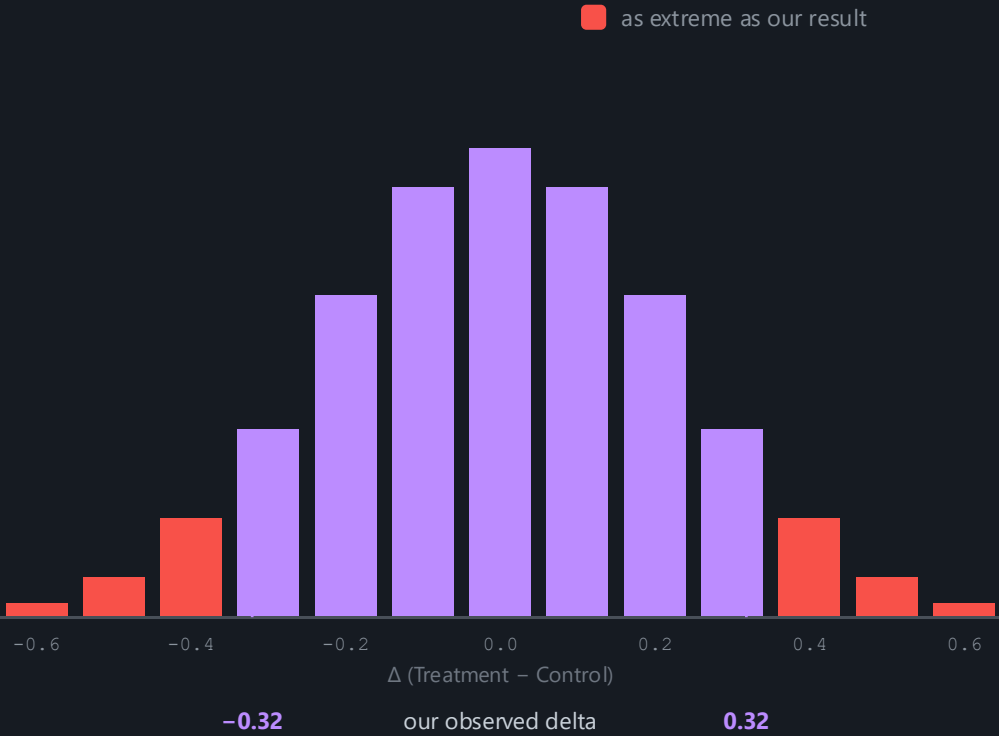
What a p-value signifies

If the treatment truly did nothing,
how unlikely is the difference we saw?

- Assume the null: the treatment had no effect.
- Then the split didn't matter — reshuffle users.
- Simulate many random splits → a spread of deltas.
- p = how often a reshuffle is as extreme as ours.
- Small p → rare by chance → likely a real effect.

$$p \approx 0.047 \quad \frac{\text{red-tail area}}{\text{total area}}$$

DELTA FROM RANDOM RE-SPLITS



In practice: we don't actually reshuffle — we use a z-test to get the same p-value, faster.



Are we just wasting time on experiments?

We could be shipping features...



Unexpected results

The symptoms

what surprised us



Wrong direction

expected tokens ↓ — but got tokens ↑



Collateral movement

seemingly unrelated metrics shifted



No movement

no significant change at all

Before trusting a surprise, ask what could have caused it →

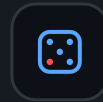
The suspects

what could cause them



Implementation error

a bug in the change itself



Type I — false positive

$100 \text{ metrics} \times p \ 0.05 \approx 5 \text{ by chance}$



Type II — false negative

a true effect stays hidden —
small effect · high variance · signal dilution



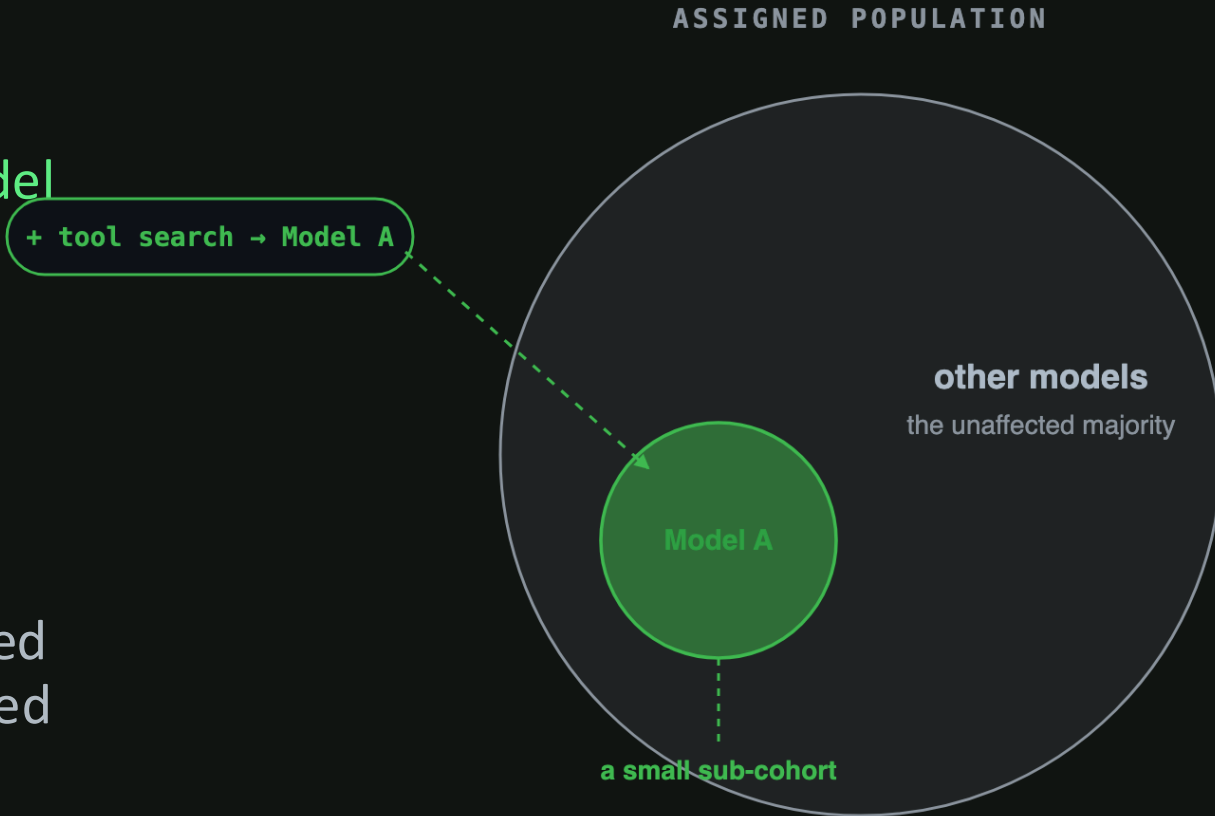
Sub-cohorts and signal dilution

We're adding a tool search support **Model A**.

We're interested in a **sub-cohort** using **Model A**.

It is a small subset of the **assigned population**.

Problem: **signal dilution** by the unaffected majority using other models => decreased sensitivity.

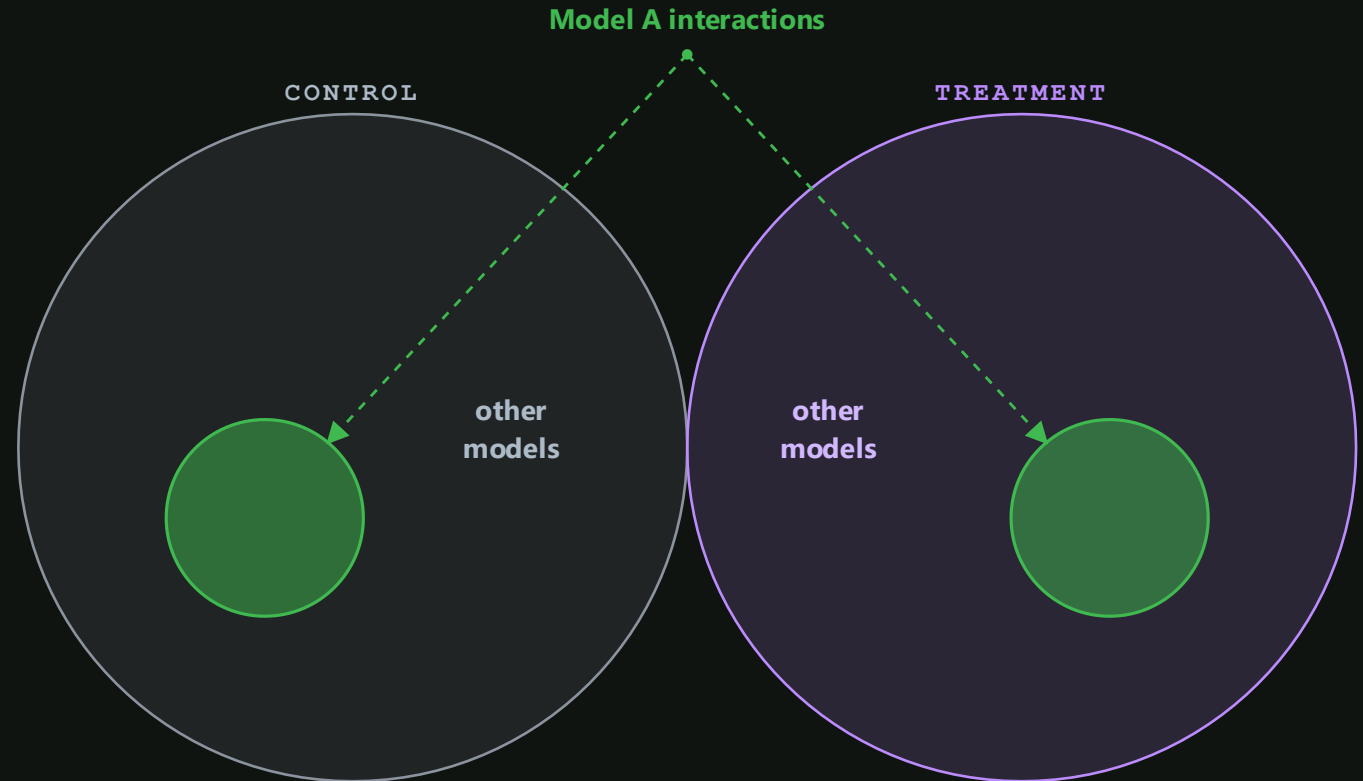


Triggered analysis

Triggered analysis gets rid of signal dilution by filtering out signals from **unaffected users**,

however

1. Smaller sample size decreases statistical power of the experiment => we need to expand the audience.
2. We may accidentally exclude eligible data points



Takeaways



MCP server

LAYER 1

Does it pick the
right tools?



Synthetic tasks

LAYER 2

Does it do the job
— before prod?



Online

LAYER 3

Does it help
real users?

- ✓ **You can't unit-test agents** — measure them statistically.
- ✓ **Isolate the testable parts** of the agent loop.
- ✓ **Pass/fail hides a lot** — watch tokens, tools, loops, and cost.
- ✓ **Be skeptical of significant movements** — they can be false positives (Type I).
- ✓ **No movement isn't proof** — could be your design, dilution, or a bug (Type II).





Thank you